ОТЧЁТ
ПО ДОМАШНЕЙ РАБОТЕ № 3

«Анализ IT-вакансий: объектное хранилище RustFS, собственный MapReduce и DuckDB»

Индивидуальный вариант № 11 - DevOps-инженер

Выполнил: Алёшкин Дмитрий Игоревич
Группа: J3311
ИСУ: 464975

2026

1. Цель и постановка задачи

Цель работы - построить локальный воспроизводимый аналитический пайплайн для вакансий DevOps-инженеров: хранение данных в S3-совместимом RustFS, независимый расчёт частот навыков средствами собственного MapReduce и SQL DuckDB, полная сверка результатов, анализ зарплат и регионов, а также измерение вычислительных затрат по времени и памяти.

Индивидуальный вариант — № 11 «DevOps-инженер». Для отбора вакансий использовано поле role с точным значением «DevOps-инженер» после удаления пробелов по краям. Это соответствует отдельной профессиональной категории исходного набора данных и позволяет независимо воспроизвести отбор в Python и SQL.

Задачи исследования:

- использовать готовый стенд RustFS из домашней работы № 2 и разместить данные в бакете hh-lake;
- подсчитать одиночные навыки и уникальные неупорядоченные пары и тройки двумя независимыми способами;
- доказать поэлементное совпадение MapReduce и DuckDB;
- сопоставить медианные зарплатные ориентиры по наборам навыков;
- сравнить три региона и выявить особенности спроса;
- измерить время и пиковую память минимум в трёх запусках, проверить программный отказ маппера и оптимизацию.

2. Испытательный стенд и инфраструктура

Таблица 1 — Аппаратно-программный стенд

Компонент	Фактическая конфигурация
ОС	Windows 11 Home, версия 10.0.26300, сборка 26300
Docker / Compose	28.3.2 / v2.39.1-desktop.1
Python	3.11.9
DuckDB	1.5.6 (расширение httpfs)
psutil	7.2.2
RustFS	1.0.0, контейнер rustfs-server
S3 API / Console	http://localhost:9000 / http://localhost:9001
Объектное хранилище	4 bind-mount каталога; EC:2 (2 data + 2 parity)
Рабочая папка	C:\rustfs-lab\devops-hw
Бакет для ДЗ № 3	hh-lake

RustFS был развёрнут в рамках ДЗ № 2 на четырёх локальных каталогах, имитирующих диски. Все каталоги расположены на одной машине, поэтому конфигурация не обеспечивает независимость физических отказов и

используется как учебный стенд. Перед расчётами контейнер запущен через Docker Compose, readiness-проверка возвратила HTTP 200.

```
cd C:\rustfs-lab
docker compose up -d --no-recreate
docker compose ps
curl.exe -sS -o NUL -w "RustFS HTTP %{http_code}\n" http://localhost:9000/health/ready
```

Контрольный вывод: контейнер rustfs-server имел статус healthy; проверка готовности RustFS возвратила HTTP 200. Исходный объект размещён в бакете hh-lake по пути raw/vacancies.csv; наличие объекта и его размер подтверждены чтением через S3 API и снимком веб-консоли. Значение SHA-256 исходного CSV унаследовано из результатов домашней работы № 2.

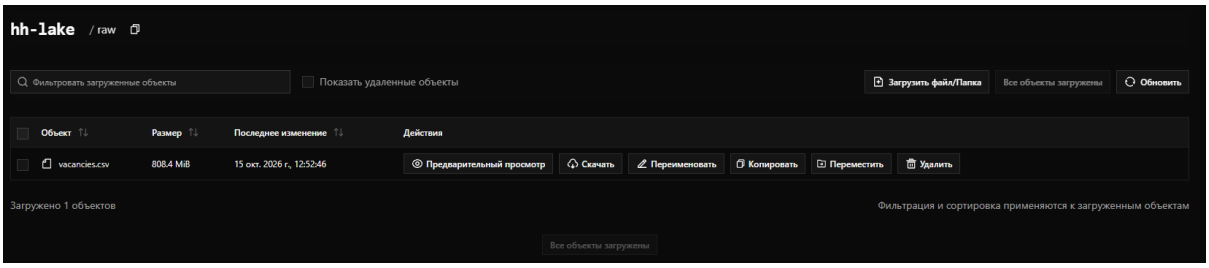


Рисунок 1 — Исходный объект vacancies.csv в бакете hh-lake/raw (консоль RustFS)

3. Исходные данные и подготовка

Таблица 2 — Контрольные характеристики набора данных

Показатель	Значение
Источник	s3://hh-lake/raw/vacancies.csv
Размер объекта	847 619 384 байта (около 808,4 МиБ)
SHA-256 исходного CSV (ДЗ № 2)	303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
Строк в CSV	229 388
Строк варианта № 11	5 158
Уникальных vacancy_id варианта	5 158
Вакансий с навыками	3 534
Вакансий без навыков	1 624
Уникальных названий навыков	1 560

Поля исходного CSV включают vacancy_id, name, role, area, key_skills, salary_from, salary_to, currency, gross, experience и другие атрибуты. Навыки представлены строкой, разделённой символами «;». Для одной вакансии удаляются пустые элементы и повторы, а комбинации упорядочиваются лексикографически. Регистры и синонимы в данной версии не нормализуются: например, Gitlab и GitLab CI остаются разными значениями.

В текущей выборке не обнаружено дублирующихся или пустых vacancy_id; внутристрочных повторений навыков также не выявлено. Наличие 1 624

вакансий без навыков учитывалось в общей численности выборки, но не в частотах сочетаний.

Базовое условие фильтрации, непосредственно записанное в SQL:

```
WHERE TRIM(COALESCE(role, '')) = 'DevOps-инженер'
```

4. Архитектура вычислительного пайплайна

Таблица 3 — Логические компоненты решения

Узел / этап	Назначение и способ работы
RustFS / raw	Хранение исходного CSV в бакете hh-lake; доступ через S3 API.
Python MapReduce	Потоковое чтение S3-объекта; фильтрация по роли; логические чанки по 250 DevOps-строк; локальные счётчики и Reduce.
DuckDB / httpfs	Независимое чтение того же S3-объекта; STRING_SPLIT, UNNEST и JOIN в SQL.
Verification	Сравнение множества ключей и частот по каждому размеру стека; отсутствие пропущенных, лишних и несовпадающих элементов.
Analytics	Медианы зарплатных ориентиров по комбинациям; сравнение Москвы, Санкт-Петербурга и Новосибирска.
Evidence	Сохранённые результаты в RustFS и журналы в локальной папке evidence.

Собственный MapReduce является последовательной локальной реализацией модели Map → Reduce, а не кластерным распределённым фреймворком. Считывание из S3 выполняется потоково; чанк означает блок из 250 отфильтрованных вакансий. Результаты мапперов агрегируются в памяти одного Python-процесса.

Для вакансии с n различными навыками число сочетаний размера k задаётся $C(n, k) = n! / (k! \times (n-k)!)$. Например, 10 навыков дают 10 одиночных записей, 45 пар и 120 троек. Этим объясняется быстрый рост объёма агрегирования при переходе от k=1 к k=3.

5. MapReduce: навыки, пары и тройки

Скрипт mapreduce_devops.py получил данные непосредственно из S3, отфильтровал вариант № 11 и выполнил 21 логический запуск маппера (20 чанков по 250 вакансий и последний чанк из 158). Для каждого чанка выполнялось построение словарей частот, после чего Reducer объединял локальные Counter.

Таблица 4 — Результаты собственного MapReduce

Размер стека	Уникальных сочетаний	Всего вхождений
1 - навык	1 560	28 344
2 - пара	22 311	134 786
3 - тройка	152 803	507 851

```
=== CONTROL VALUES ===
DevOps vacancies: 5158
Vacancies with skills: 3534
Mapper executions: 21
Stack 1: 1560 unique combinations, 28344 occurrences
Stack 2: 22311 unique combinations, 134786 occurrences
Stack 3: 152803 unique combinations, 507851 occurrences
CONTROL CHECK: PASSED
```

Объекты результатов записаны в s3://hh-lake/results/mapreduce/skills_1.csv, skills_2.csv и skills_3.csv. Их размеры: 33 091, 685 676 и 6 063 651 байт соответственно. Сумма числа вхождений не равна количеству вакансий, поскольку каждая вакансия формирует множество сочетаний.

Таблица 5 — Наиболее частые навыки DevOps-вакансий

Навык	Частота	Навык	Частота
Linux	2 190	Kubernetes	1 831
Docker	1 812	DevOps	1 288
Ansible	1 232	Python	1 058
CI/CD	1 020	Bash	925
PostgreSQL	892	Grafana	823
Git	733	Jenkins	662
Prometheus	660	Nginx	551
Gitlab	546	Zabbix	480
Terraform	462	ELK	435

6. Независимая реализация SQL в DuckDB

Вторая реализация использовала DuckDB 1.5.6 и расширение httpfs для прямого чтения s3://hh-lake/raw/vacancies.csv. После самостоятельного условия выбора DevOps-строк SQL разделял key_skills с помощью STRING_SPLIT, преобразовывал массив в строки через UNNEST и использовал DISTINCT для учёта уникальных навыков в вакансии.

Для пар и троек применялись самостоятельные JOIN по vacancy_id. Условия a.skill < b.skill и b.skill < c.skill исключали перестановки одной комбинации и повторяющиеся элементы.

```
-- Фрагмент SQL формирования пар
SELECT a.skill AS skill_1, b.skill AS skill_2, COUNT(*) AS count
FROM devops_skills AS a
JOIN devops_skills AS b
  ON a.vacancy_id = b.vacancy_id AND a.skill < b.skill
GROUP BY a.skill, b.skill;
```

Таблица 6 — Поэлементная верификация результатов DuckDB и MapReduce

Проверка	Одиночные	Пары	Тройки
Уникальных ключей	1 560	22 311	152 803
Сумма частот	28 344	134 786	507 851
Недостающих ключей	0	0	0
Лишних ключей	0	0	0
Разных частот	0	0	0

Comparison stack 1: missing=0, extra=0, different_counts=0
Stack 1: EXACT MATCH
Comparison stack 2: missing=0, extra=0, different_counts=0
Stack 2: EXACT MATCH
Comparison stack 3: missing=0, extra=0, different_counts=0
Stack 3: EXACT MATCH
DUCKDB VERIFICATION PASSED

Всего по трём уровням сверено 176 674 различных ключа. Результаты DuckDB сохранены в s3://hh-lake/results/duckdb/skills_1.csv, skills_2.csv и skills_3.csv. Таким образом подтверждено не только совпадение контрольных сумм, но и равенство каждой частоты на всех трёх уровнях.

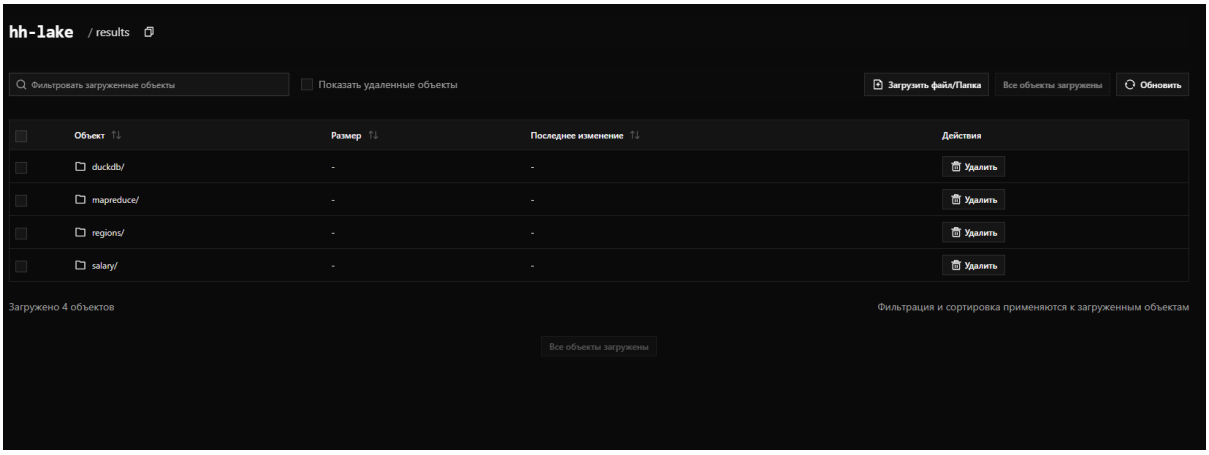


Рисунок 2 — Каталоги результатов MapReduce, DuckDB, зарплатного и регионального анализа в hh-lake/results

7. Анализ зарплатного ориентира

7.1. Качество зарплатных данных

Таблица 7 — Покрытие зарплатными данными

Показатель	Вакансий
Всего DevOps-вакансий	5 158
Есть хотя бы одна граница зарплаты	1 111
RUR (включая gross и net)	1 037
RUR, gross=false	762
RUR, gross=false и есть навыки	564
Указаны обе границы (все валюты)	435
Только нижняя граница (все валюты)	398
Только верхняя граница (все валюты)	278

Для сопоставимости основной анализ ограничен суммами в RUR/RUB с gross=false, то есть вакансиями с оплатой после вычета налогов. Значения USD, EUR, BYR и gross=true исключены без произвольного пересчёта по валютным и налоговым коэффициентам. Для двух заданных границ использована середина диапазона; для единственной указанной границы -

сама эта граница. Полученная переменная является зарплатным ориентиром, а не фактически выплачиваемым доходом.

Из 762 подходящих предложений у 292 есть две границы, у 289 - только нижняя, у 181 - только верхняя. Медианный зарплатный ориентир равен 210 000 ₽.

7.2. Базовые группы и сравнение

Для пары базой служил более частый одиночный навык во всех 5 158 DevOps-вакансиях; для тройки — наиболее частая из входящих в неё пар. В таблицах n — число зарплатных наблюдений в комбинации, n базы — число таких наблюдений в базовой группе. Изменение вычислялось относительно медианы базовой группы как в рублях, так и в процентах. Применены диагностические пороги: n пары ≥ 10, n тройки ≥ 8 и n базы ≥ 15; они не являются тестом статистической значимости.

Таблица 8 — Примеры положительных и отрицательных отклонений для пар

Комбинация / базовый навык	n	n базы	Медиана, ₽	База, ₽	Δ, ₽ (%)
DevOps + Terraform / DevOps	44	251	287 500	200 000	+87 500 (43,8%)
GitLab CI + Linux / Linux	15	365	300 000	200 000	+100 000 (50%)
Kubernetes + OpenShift / Kubernetes	12	291	350 000	250 000	+100 000 (40%)
Debian + Linux / Linux	17	365	100 000	200 000	−100 000 (−50%)
Ansible + Atlassian Jira / Ansible	10	199	125 000	250 000	−125 000 (−50%)
PostgreSQL + СУБД / PostgreSQL	15	199	125 000	250 000	−125 000 (−50%)

Таблица 9 — Примеры положительных и отрицательных отклонений для троек

Тройка / базовая пара	n	n базы	Медиана, ₽	База, ₽	Δ, ₽ (%)
Git + Gitlab + Python / Git + Python	17	53	350 000	180 000	+170 000 (94,4%)
CI/CD + Python + Zabbix / CI/CD + Python	15	53	350 000	200 000	+150 000 (75%)
Docker-compose + Linux + Prometheus / Linux + Prometheus	8	104	100 000	250 000	−150 000 (−60%)
Bash + Debian + Linux / Bash + Linux	8	120	100 000	235 000	−135 000 (−57,4%)

Всего рассчитано 564 группы одиночных навыков с зарплатой, 7 526 пар и 51 778 троек. Установленным порогам для сравнения отвечали 440 пар и 2 226 троек. Сочетание DevOps + Terraform основано на 44 зарплатных предложениях, что делает его более устойчивым описательным наблюдением, чем экстремальные тройки с n=8–9. Наличие положительного или отрицательного Δ не означает причинного влияния навыка на зарплату.

Возможные источники смещения: различия по опыту и региону, неслучайное отсутствие зарплат, неполные диапазоны, различия терминологии навыков и малочисленные комбинации. Для строгого вывода о влиянии навыка потребовалась бы модель с контролем сопутствующих факторов и проверкой устойчивости на подвыборке полных зарплатных диапазонов.

8. Сравнение трёх регионов

Для описательного сравнения выбраны Москва, Санкт-Петербург и Новосибирск: 3 007, 686 и 160 DevOps-вакансий соответственно. Зарплатный анализ регионов использует то же определение рублёвого net-ориентира, что и общий зарплатный раздел.

Таблица 10 — Региональные показатели

Показатель	Москва	Санкт-Петербург	Новосибирск
Вакансии	3 007	686	160
Вакансии с навыками	2 015	488	123
Зарплатные наблюдения, n	406	103	41
Медиана net-ориентира, ₽	250 000	220 000	165 000
Покрытие зарплатами	13,50%	15,01%	25,63%
Полные диапазоны, n	161	36	15
Медиана полных диапазонов, ₽	250 000	227 500	40 000

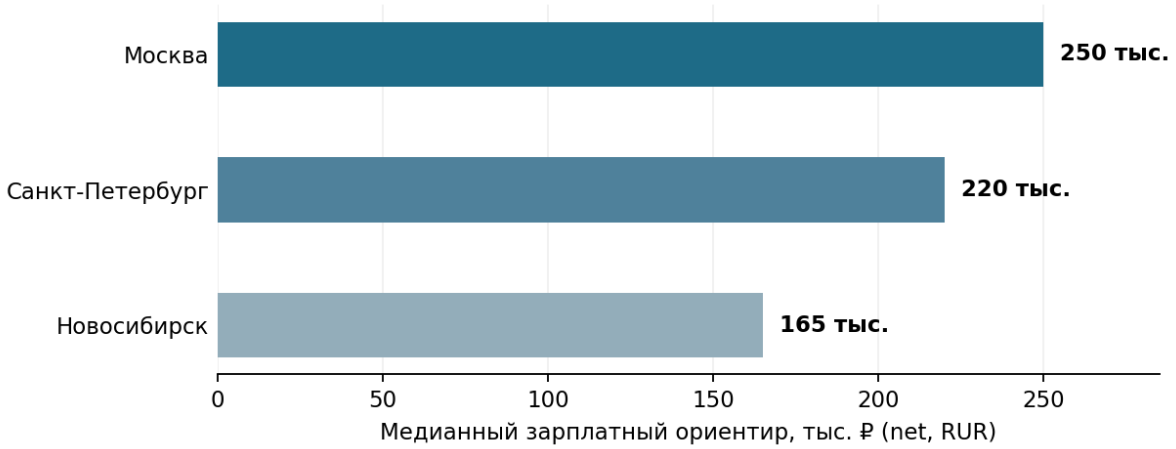


Рисунок 3 — Медианный зарплатный ориентир DevOps-вакансий по регионам

Таблица 11 — Частота распространённых навыков и тройки (доля всех вакансий региона)

Регион	Linux	Docker	Kubernetes	Docker + Kubernetes + Linux
Москва	1 210 (40,24%)	980 (32,59%)	1 085 (36,08%)	425 (14,13%)
Санкт-Петербург	299 (43,59%)	274 (39,94%)	252 (36,73%)	109 (15,89%)
Новосибирск	86 (53,75%)	70 (43,75%)	70 (43,75%)	44 (27,50%)

Таблица 12 — Три наиболее частые пары навыков по регионам (доля всех вакансий)

Регион	Пара навыков	Вакансий	Доля, %
Москва	Docker + Kubernetes	681	22,65
Москва	Kubernetes + Linux	650	21,62
Москва	Docker + Linux	630	20,95
Санкт-Петербург	Docker + Kubernetes	173	25,22
Санкт-Петербург	Docker + Linux	172	25,07
Санкт-Петербург	Kubernetes + Linux	153	22,30
Новосибирск	Docker + Linux	54	33,75
Новосибирск	Kubernetes + Linux	54	33,75
Новосибирск	Docker + Kubernetes	53	33,12

Москва имеет максимальное число DevOps-вакансий и наибольший медианный ориентир (на 30 000 ₽ выше Санкт-Петербурга и на 85 000 ₽ выше Новосибирска). Во всех трёх регионах преобладают Linux, Docker, Kubernetes и их сочетания. Однако относительные доли различаются: тройка Docker + Kubernetes + Linux встречается в 27,50% вакансий Новосибирска против 14,13% в Москве; пара Docker + Linux — в 33,75% против 20,95%. Это наблюдение о структуре объявлений, а не доказательство различий квалификации или причин зарплатных расхождений.

В Новосибирске медианный ориентир по всем 41 сопоставимым зарплатным предложениям составляет 165 000 ₽, а медиана 15 вакансий с полными диапазонами — 40 000 ₽. Существенное различие связано с неодинаковым составом двух подвыборок и чувствительностью результата к типу заполнения зарплаты. Поэтому для межрегионального сравнения использована основная медиана по единой методике, а медиана полных диапазонов показана отдельно как диагностический показатель, не характеризующий весь рынок.

9. Сравнение производительности

Базовое профилирование выполнено в Windows 11 на одинаковом исходном CSV, читаемом из RustFS. DuckDB запущен с threads=1. Для каждого подхода выполнен один прогревочный запуск и три измеряемых запуска в чередующемся порядке. В сравнение входит чтение из S3 и вычисление агрегатов, без сохранения результирующих объектов в S3.

Таблица 13 — Профилирование исходных методов (три измеряемые попытки)

Попытка	MapReduce, с	DuckDB, с	MapReduce peak, МиБ	DuckDB peak, МиБ
1	11,783	9,752	87,91	1 298,08
2	11,483	9,433	87,97	1 298,09
3	11,332	9,356	87,77	1 297,89
Медиана времени	11,483	9,433	-	-
Максимальный пик памяти	-	-	87,97	1 298,09

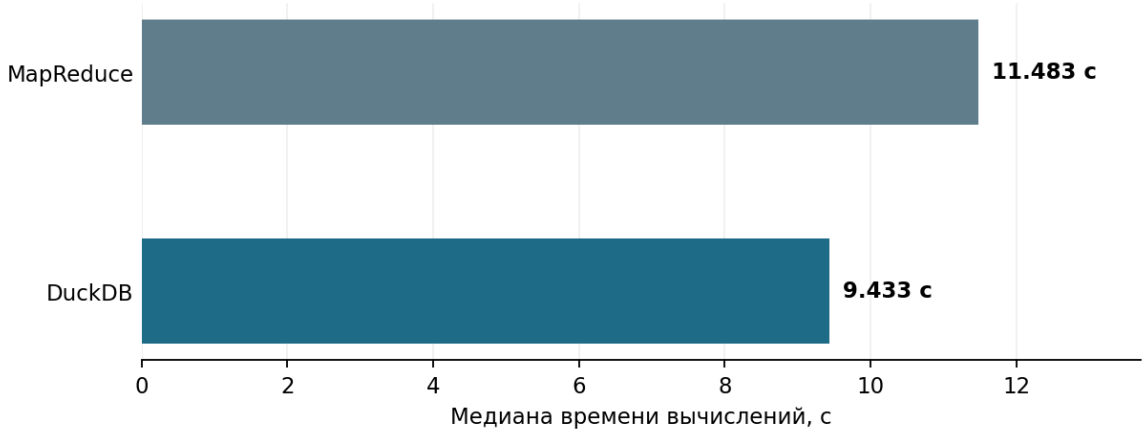


Рисунок 4 — Сравнение медианного времени вычислений

В данной конфигурации DuckDB был быстрее: 11,483 / 9,433 ≈ 1,22 раза; время меньше на 17,9%. В то же время пик рабочего набора процесса DuckDB (1 298,09 МиБ) примерно в 14,8 раза выше MapReduce (87,97 МиБ). Поэтому подходы демонстрируют компромисс скорости и памяти; результаты не следует переносить без измерений на другие машины, число потоков и объёмы данных.

Метрика памяти - Windows Peak Working Set процесса, полученная внутри рабочего Python-процесса с помощью psutil (peak_wset). Первоначальные значения около 4 МиБ, снятые опросом внешнего процесса, признаны недостоверными и исключены из окончательных таблиц. Показатель не включает память отдельного контейнера RustFS и не тождественен суммарной физической памяти всей системы.

10. Тестирование сбоя маппера

В отдельном тесте имитировался сбой логического маппера № 5 после локального Map и до фиксации его результатов в Reduce. Первый вызов завершался специально вызванным исключением; повторная попытка успешно обрабатывала тот же чанк. Частичные результаты фиксировались только после успешного выполнения; двойного учёта не возникло.

```
MAPPER 5 FAILED on attempt 1: Injected failure after map calculation
Retrying mapper 5...
MAPPER 5 RECOVERED on attempt 2
Mapper chunks processed: 21
DevOps vacancies processed: 5158
Recovered mapper chunks: 1
Stack 1: EXACT MATCH | 1560 unique | 28344 occurrences
Stack 2: EXACT MATCH | 22311 unique | 134786 occurrences
Stack 3: EXACT MATCH | 152803 unique | 507851 occurrences
```

Дополнительно проверен постоянный программный отказ маппера № 999: обе попытки завершились ошибкой, превышение лимита было обнаружено,

неуспешный результат не добавлялся к агрегату. Финальный статус - FAILURE RECOVERY TEST PASSED. Эксперимент не моделирует аппаратный отказ узла или сбой сети распределённого кластера.

11. Бонус: оптимизация повторных расчётов

В качестве оптимизации подготовлен компактный объект s3://hh-lake/curated/devops_skills.csv, содержащий только vacancy_id и key_skills для 5 158 DevOps-вакансий. Время разового чтения полного CSV, выделения выборки, загрузки и проверки подготовленного объекта составило 10,055 с. Размер нового объекта - 369 794 байта; SHA-256 - 541174a6d1f5a80a949dfa7a4255934ea3659a6aa850c6e859892a5a6c2e9093.

Результаты повторного вычисления для 1 560 одиночных навыков, 22 311 пар и 152 803 троек сравнивались с эталонными CSV исходного MapReduce и совпали по каждому ключу и частоте.

Таблица 14 — Профилирование до и после подготовки DevOps-выборки

Попытка	До, с	После, с	До, МиБ	После, МиБ
1	11,701	0,737	88,12	85,92
2	11,730	0,740	88,46	85,75
3	12,744	0,535	87,72	86,73
Медиана времени	11,730	0,737	-	-
Максимальный пик	-	-	88,46	86,73

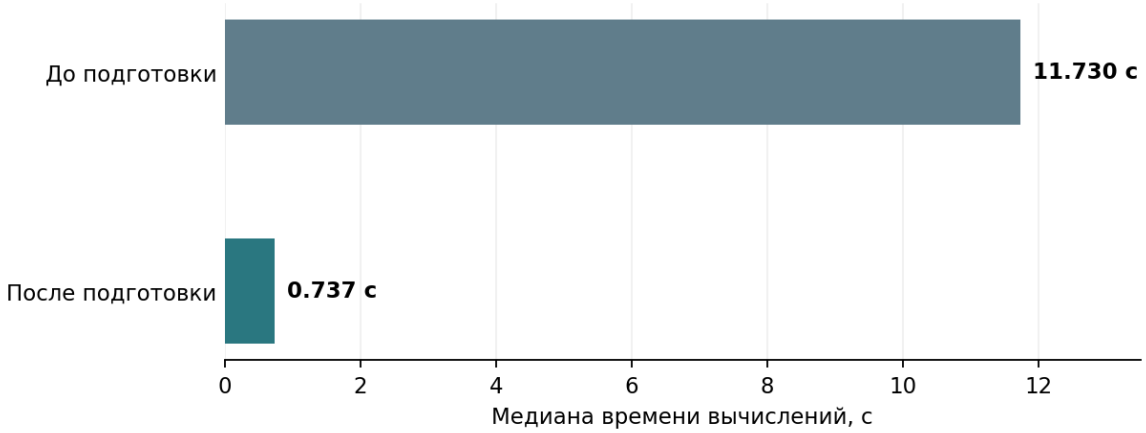


Рисунок 5 — Ускорение повторного MapReduce при использовании curated-объекта

Ускорение повторного расчёта по медиане времени - 15,92 раза; снижение времени - 93,72%. Максимальный пик памяти сократился на 1,96%. Разовая стоимость подготовки данных - 10,055 с, экономия на одном повторном запуске по медианам - 10,993 с; следовательно, затраты на подготовку окупаются уже при одном последующем расчёте в рассматриваемом режиме.

Важно: сравнение «до/после» относится к разным объёмам входных данных: полному CSV и предварительно отфильтрованной выборке. Это оптимизация ETL-пайплайна при повторной аналитике, а не ускорение внутреннего алгоритма Map/Reduce на одном и том же входном файле. Для одиночного нерегулярного расчёта следует включать стоимость подготовки в общие затраты.

12. Воспроизводимость и состав проекта

Таблица 15 — Основные программные компоненты, подтверждённые проверкой состава каталога

Группа	Исходные файлы / результат
Инфраструктура и данные	check_storage.py; copy_dataset.py; inspect_vacancies.py; profile_devops.py
MapReduce и проверка	mapreduce_devops.py; test_mapper_recovery.py
DuckDB / SQL	duckdb_devops.py; sql/prepare.sql; sql/skills_1.sql; sql/skills_2.sql; sql/skills_3.sql
Аналитика	salary_audit.py; salary_insights.py; regional_analysis.py; sql/salary_*.sql; sql/regional_*.sql
Профилирование	benchmark_compute.py; benchmark_optimization.py
Оптимизация	optimized_mapreduce.py
Доказательства	evidence/*.txt, evidence/*.csv; результаты в RustFS

По результатам проверки локального каталога сохранены 13 основных Python-программ, 12 SQL-файлов, выходные CSV и журналы испытаний. Программы охватывают подключение к RustFS, подготовку данных, MapReduce, DuckDB, зарплатный и региональный анализ, профилирование, тесты отказа маппера и оптимизацию.

Команды воспроизведения расчётов на подготовленном локальном стенде:

```
cd C:\rustfs-lab
docker compose up -d --no-recreate
cd C:\rustfs-lab\devops-hw
.\.venv\Scripts\python.exe .\check_storage.py
.\.venv\Scripts\python.exe .\mapreduce_devops.py
.\.venv\Scripts\python.exe .\duckdb_devops.py
.\.venv\Scripts\python.exe .\salary_insights.py
.\.venv\Scripts\python.exe .\regional_analysis.py
.\.venv\Scripts\python.exe .\benchmark_compute.py
.\.venv\Scripts\python.exe .\test_mapper_recovery.py
.\.venv\Scripts\python.exe .\benchmark_optimization.py
```

13. Итоговые выводы

- Локальный RustFS после восстановления из ДЗ № 2 успешно работал с S3 API; исходный CSV объёмом 847 619 384 байта доступен в бакете hh-lake.

- Для варианта № 11 выделены 5 158 DevOps-вакансий. Собственный потоковый MapReduce сформировал 1 560 навыков, 22 311 пар и 152 803 тройки; все результаты размещены в объектном хранилище.
- DuckDB самостоятельно обработал те же данные и обеспечил точное совпадение всех 176 674 комбинаций с MapReduce, включая индивидуальные частоты.
- В зарплатном анализе медианный net-ориентир по 762 рублёвым вакансиям составил 210 000 ₽. Наблюдаемые изменения медиан зависят от состава выборки и не интерпретируются как причинное действие навыков.
- Медианный net-ориентир: Москва - 250 000 ₽, Санкт-Петербург - 220 000 ₽, Новосибирск - 165 000 ₽. Тройка Docker + Kubernetes + Linux относительно чаще встречалась в Новосибирске (27,50%).
- В базовом профилировании DuckDB оказался в 1,22 раза быстрее по медиане времени, но потребовал в 14,8 раза больше пиковой памяти вычислительного процесса.
- Моделирование сбоя пятого маппера и постоянного отказа дополнительного маппера завершилось ожидаемым восстановлением/отказом; корректность агрегатов подтверждена.
- Подготовка компактной DevOps-выборки ускорила повторный MapReduce в 15,92 раза при сохранении идентичности результатов. Это подтверждает эффективность предварительной фильтрации для повторной аналитики.